

A feladat teljes megoldását egyetlen .cs fájlba helyezze el. A fájl elnevezése a feladat azonosítója és az Ön saját Neptun kódja legyen, alulvonással elválasztva, nagybetűkkel: **AZONOSÍTÓ_NEPTUNKOD.cs**

A feladattal kapcsolatos további információk az utolsó oldalon találhatók. Ezen ismeretek hiányából adódó reklamációt nem fogadunk el.

A klasszikus dominóban két játékos felváltva építi a sort. A játék egyszemélyes változatában a játékos célja a rendelkezésére álló összes dominót felhasználva kirakni a sort.

Készítsen paracssori alkalmazást, amely meghatározza, hogy a rendelkezésre álló N darab dominó mindegyikét felhasználva lehetséges-e olyan útvonal létrehozása, amely összefüggő és nem tartalmaz elágazást (igen vagy nem, "Y" vagy "N").

Bemenet (Console)

- *első sor* - a dominók (N) száma
- *további N sor* - a dominón található A és B szám | karakterrel elválasztva

Kimenet (Console)

- a Console-on megjelenítve vagy az "Y" (ha lehetséges létrehozni az útvonalat az összes dominót felhasználva), vagy az "N" karakter (ha nem)

Megkötés(ek)

- $1 \leq N \leq 100$
- $0 \leq A, B \leq 6$

Megjegyzés

- a dominók szükség szerint megfordíthatók, illetve átrendezhetők

Példa

Console input	Console output
2 1 6 6 3	Y

Értelmezés

Az első sor alapján ($N=2$) darab dominót kell megpróbálnunk sorba lerakni. Az első dominón az 1 | 6, a másodikon a 6 | 3 számok találhatók.

Ezeket egymás mellé tudjuk helyezni, így lehetséges olyan sort létrehozni, amiben minden dominót felhasználtunk, vagyis az Y karaktert kell megjeleníteni a felhasználónak.

Tesztesetek

Az alkalmazás helyes működését legalább az alábbi bemenetekkel tesztelje le!

1.	Console input 2 1 6 6 3	Console output Y
2.	Console input 2 1 6 3 6	Console output Y
3.	Console input 2 1 1 1 1	Console output Y

4.

Console input

```
3
1 | 1
2 | 2
3 | 3
```

Console output

N

5.

Console input

```
5
0 | 1
1 | 2
2 | 3
3 | 4
4 | 5
```

Console output

Y

6.

Console input

```
3
0 | 1
1 | 2
3 | 4
```

Console output

N

7.

Console input

```
20
0 | 1
2 | 3
5 | 6
1 | 2
6 | 4
5 | 0
3 | 4
4 | 2
2 | 1
4 | 5
1 | 3
3 | 5
0 | 5
5 | 2
0 | 3
3 | 3
2 | 6
6 | 1
1 | 4
4 | 0
```

Console output

Y

8.

Console input

```
3
1 | 6
6 | 3
6 | 4
```

Console output

N

A fenti tesztesetek nem feltétlenül tartalmazzák az összes lehetséges állapotát a be- és kimenet(ek)nek, így saját tesztekkel is próbálja ki az alkalmazás helyes működését!

Tájékoztató

A feladattal kapcsolatosan általános szabályok:

- A feladat megoldását egy Console App részeként kell elkészíteni, "Do not use top-level statements" beállítással, illetve a szükséges "using"-ok megtartása mellett.
- Megoldásként csak a Program.cs forrásfájl kell beadni, amelynek elnevezése a feladat azonosítója és a szerző saját Neptun kódja legyen alulvonással elválasztva, nagy betűkkel: **AZONOSÍTÓ_NEPTUNKOD.cs**
- A megvalósítás során lehetőség szerint alkalmazza az előadáson és a laboron ismertetett algoritmusokat, figyelembe véve a *Megkötések* pontban definiáltakat. Ezeket leszámítva viszont legyen kreatív a feladat megoldásával kapcsolatban.
- Az alkalmazás elkészítése során minden esetben törekedjen a megfelelő típusok használatára, illetve az igényes (*formázott, felesleges változóktól és utasításoktól mentes*) kód kialakítására, amely magába foglalja az elnevezésekkel kapcsolatos ajánlások betartását is (*bővebben*).
- **Ne másoljon vagy adja be más megoldását!** Minden ilyen esetben az összes (felépítésben) azonos megoldás duplikátumként lesz megjelölve és a megoldás el lesz utasítva.
- **A határidő után leadott vagy helytelen elnevezésű megoldás, vagy a kiírásnak nem megfelelő megoldás, vagy fordítási hibát tartalmazó, vagy (helyes bemenetet megadva) futásidejű hibával leálló kód nem értékelhető!**
- A feladat leírása az alábbiak szerint épül fel (* - opcionális):
 - *Feladat leírása* - a feladat megfogalmazása
 - *Bemenet* - a bemenettel kapcsolatos információk
 - *Kimenet* - az elvárt kimenettel kapcsolatos információk
 - *Megkötések* - a bemenettel, a kimenettel és az algoritmussal kapcsolatos megkötések, melyek figyelembevétele és betartása kötelező, továbbá az itt megfogalmazott bemeneti korlátoknak a tesztek minden esetében eleget tesznek, így olyan esetekre nem kell felkészülni, amik itt nincsenek definiálva
 - **Megjegyzések* - további, a feladattal, vagy a megvalósítással kapcsolatos megjegyzések
 - *Példa* - egy példa a feladat megértéséhez
 - *Tesztesetek* - további tesztesetek az algoritmus helyes működésének teszteléséhez, mely nem feltétlenül tartalmazza az összes lehetséges állapotát a be- és kimenet(ek)nek
- **Minden esetben pontosan azt írja ki és olvassa be az alkalmazás, amit a feladat megkövetel, mivel a megoldás kiértékelése automatikusan történik!** Így például, ha az alkalmazás azzal indul, hogy kiírja a konzolra a "Kérem a számot:" üzenetet, akkor a kiértékelés sikertelen lesz, a megoldás hibásnak lesz megjelölve, ugyanis egy számot kellett volna beolvasni a kiírás helyett.
- A kiértékelés során csak a *Megkötések* pont szerinti helyes bemenettel lesz tesztelve az alkalmazás, a "tartományokon" kívüli értéket nem kell lekezelnie az alkalmazásnak.
- Elősegítve a fejlesztést, a beadott megoldás utolsó utasításaként szerepelhet egyetlen Console.ReadLine() metódushívás.
- Az automatikus kiértékelés négy részből áll:
 - Unit Test-ek - az alkalmazás futásidejű működésének vizsgálatára
 - Szintaktikai ellenőrzés - az alkalmazás felépítésének vizsgálatára
 - Tervezési irányelvek ellenőrzése - az alkalmazás "kinézetének" vizsgálatára
 - Duplikációk keresése - az azonos megoldások kiszűrésére
 - Metrikák meghatározása - tájékoztató jelleggel
- A kiértékelések eredményéből egy HTML report generálódik, melyet minden hallgató megismerhet.
- A feladat megoldásának minden esetben fordíthatónak és futtathatónak kell lennie **C# 10** és **.NET 6** keretrendszer használatával. Ettől függetlenül az elkészítés során használható egyéb változata a .NET keretrendszernek és a C# nyelvnek, azonban leadás előtt győződjön meg róla, hogy a megoldása kompatibilis a .NET 6 és C# 10 verzióval.

- A keretrendszer mellett további általános, nyelvi elemekkel való megkötés, melyek a házi feladatok során nem használhatók a megoldásában:
 - LINQ: `System.Linq` - *all query expressions within the namespace*
 - Attributes
 - Collections: `ArrayList`, `BitArray`, `DictionaryEntry`, `Hashtable`, `Queue`, `SortedList`, `Stack`
 - Keywords:
 - Modifiers: `abstract`, `async`, `event`, `external`, `in` - *generic modifier*, `new` - *member modifier*, `out` - *generic modifier*, `sealed`, `unsafe`, `virtual`, `volatile`
 - Statement: `break` - *in a loop*, `continue`, `goto`, `throw`, `try-catch-finally`, `checked`, `unchecked`, `fixed`, `lock`, `yield`
 - Namespace: `extern alias`
 - Generic type constraint: `new`, `where`
 - Access: `base`
 - Contextual: `add`, `partial` - *type, method*, `remove`, `required`, `when` - *filter condition*,
 - Query: `from`, `where`, `select`, `group`, `into`, `orderby`, `join`, `let`, `in`, `out`, `equals`, `by`, `ascending`, `descending`
 - Operators and Expressions:
 - Null-conditional operators: `?.` - *null conditional member access*, `[]` - *null conditional element access*
 - User-defined conversion operators: `implicit`, `explicit`
 - Pointer: `*` - *pointer*, `&` - *address-of*, `*` - *pointer indirection*, `->` - *pointer member access*, `[]` - *pointer element access*, `+`, `-`, `++`, `--` - *pointer arithmetic operators*
 - Assignment: `ref`
 - Lambda: `=>` - *expression, statement*
 - Others: `!` - *null forgiving*, `??` - *null coalescing*, `??=` - *null coalescing assignment*, `::` - *namespace alias qualifier*, `await`, `default` - *operator, literal*, `delegate`, `sizeof`, `stackalloc`, `with` - *expression, operator*
 - Types: `dynamic`, `interface`, `object`, `Object`, `var`, `struct`, `nullable`, `pointer`, `record`, `Tuple`, `Func<T>`, `Action<T>`, `Expression<T>`, `Nullable<T>`, `Span<T>`
 - Preprocessor directives: `#nullable`, `#if`, `#elif`, `#else`, `#endif`, `#define`, `#undefine`, `#undef`, `#error`, `#warning`, `#line`, `#pragma`
- Névterek, melyek kizárólagosan importálhatók a megoldásban (minden további névtér import törlésre kerül, illetve, ha az alábbiak közül valamelyik hiányzik, akkor hozzáadásra kerül a megoldáshoz):
 - `System`
 - `System.Collections.Generic`
 - `System.IO`
 - `System.Threading`